# Visualização Geométrica, Rasterização e Recorte de Árvores Arteriais CCO

**João Pedro Silva Cruz**[1], **Pedro Vítor Silva Cruz**[1]

[1]Departamento de Computação (DECOM) — Universidade Federal de Ouro Preto (UFOP)

BCC327 — Computação Gráfica — 2026.2 — Trabalho Prático 1

{joao.prsc@aluno.ufop.edu.br, pedro.vsc@aluno.ufop.edu.br}

**RESUMO**

**Contexto:** Modelos de árvores arteriais geradas pelo método *Constrained Constructive Optimization* (CCO) são descritos por pontos, segmentos e um raio por vaso, e servem de cena para estudar os algoritmos clássicos de Computação Gráfica. **Objetivo:** Construir uma aplicação que leia modelos CCO em formato VTK e os visualize, transforme, rasterize e recorte, deixando explícito o que é implementação própria e o que vem de bibliotecas (trabalho prático 1 da disciplina BCC327). **Metodologia:** Leitura e janela 3D com PyVista/VTK; matrizes de modelo, vista e projeção, Bresenham, *Scanline Fill*, *supersampling* e os recortes de Cohen–Sutherland, Liang–Barsky, Cyrus–Beck e Sutherland–Hodgman implementados em Python e NumPy; interface em PyQt5. **Resultados:** Os algoritmos foram validados por autotestes aleatórios (por exemplo, 20 000 segmentos nos quais os três recortes de segmentos coincidem; 600 polígonos no Sutherland–Hodgman) e demonstrados em exemplos numéricos; na cena de teste, o *supersampling* $4 \times 4$ reduziu o erro quadrático médio de 2414,0 para 54,1. **Conclusão:** A aplicação integra os sete módulos pedidos; o caso do polígono côncavo em U evidencia a limitação do Sutherland–Hodgman e o motivo de se usar o Weiler–Atherton.

**Palavras-Chave:** Computação Gráfica; Método CCO; Rasterização; Antialiasing; Recorte; VTK.

## 1 Introdução

Modelos computacionais de árvores vasculares são usados para estudar a geometria e a perfusão de órgãos. O método CCO constrói esses modelos adicionando segmentos a uma árvore de modo a minimizar um custo sujeito a restrições [1, 2]. O resultado é uma estrutura geométrica simples: pontos no plano ou no espaço, segmentos de reta que os ligam e um atributo por vaso, o raio.

O objetivo deste trabalho não é gerar essas árvores (os modelos 2D e 3D foram fornecidos pelo professor), e sim usá-las como cena para aplicar e comparar os algoritmos clássicos de Computação Gráfica. Os objetivos específicos, um por módulo do enunciado, são:

1. ler os arquivos VTK e apresentar número de pontos e segmentos, raio mínimo e máximo, primitivas e atributos visuais (Módulo 1);
2. aplicar translação, rotação e escala, posicionar o observador em várias vistas e comparar as projeções ortográfica e perspectiva (Módulo 2);
3. rasterizar segmentos com Bresenham (Módulo 3) e preencher polígonos por *Scanline* (Módulo 4);
4. comparar a imagem com e sem antialiasing (Módulo 5);
5. recortar segmentos (Módulo 6) e polígonos (Módulo 7).

O trabalho também documenta, de forma explícita, o que foi implementado pelo grupo e o que veio das bibliotecas (Seção 3.9.6).

## 2 Trabalhos Relacionados

O método CCO foi introduzido por Schreiner e Buxbaum [1] e estendido a árvores tridimensionais por Karch et al. [2]. Neste trabalho essas árvores são apenas o dado de entrada: os modelos 2D e 3D foram fornecidos pelo professor, e o método de geração não foi reimplementado. Do lado da Computação Gráfica, os algoritmos aplicados são clássicos e estão descritos em livros-texto [8]; a visualização interativa usa o VTK [9] por meio do PyVista [10]. A Tabela 1 resume o papel de cada referência no trabalho.

Tabela 1: Referências principais e seu uso neste trabalho.

| **Tópico** | **Referência** | **Uso neste trabalho** |
|---|---|---|
| Geração de árvores CCO | Schreiner e Buxbaum (1993); Karch et al. (1999) | Origem dos modelos VTK 2D e 3D; método não reimplementado |
| Rasterização de linhas | Bresenham (1965) | Implementado (8 octantes, aritmética inteira); DDA para comparação |
| Recorte de segmentos | Cohen–Sutherland [8]; Liang e Barsky (1984); Cyrus e Beck (1978) | Os três implementados e comparados |
| Recorte de polígonos | Sutherland e Hodgman (1974); Weiler e Atherton (1977) | Sutherland–Hodgman implementado; Weiler–Atherton apenas discutido |
| Visualização 3D | VTK; PyVista | Leitura do arquivo, janela OpenGL, tubos e antialiasing do VTK |

## 3 Fundamentação Teórica e Metodologia

### 3.1 Árvores CCO e o arquivo VTK

No CCO, a árvore cresce por inserção sucessiva de terminais; cada vaso é um segmento com raio próprio [1]. A versão 3D, usada em [2], representa a árvore no espaço. Para este trabalho basta o formato geométrico: o arquivo VTK guarda as coordenadas dos pontos, as células do tipo linha (conectividade) e arrays de atributos, entre eles o raio. Uma célula de linha pode ser uma polilinha com $n$ pontos, que equivale a $n-1$ segmentos.

### 3.2 Modelo de cor

O raio de cada vaso é codificado por um *mapa contínuo de cores* (*viridis*), com normalização linear $t = (r - r_{\min})/(r_{\max} - r_{\min}) \in [0, 1]$. Essa escolha foi preferida a um mapeamento RGB direto por três razões: o raio é uma grandeza escalar, e um mapa contínuo a representa por uma única variável; o *viridis* é perceptualmente uniforme, ou seja, passos iguais em $t$ parecem diferenças de cor parecidas; e ele continua legível em escala de cinza e para daltonismo. A barra de cores mostra a correspondência entre cor e raio. Além da cor, a espessura também carrega o raio: nos tubos, o raio geométrico é proporcional ao atributo.

### 3.3 Espaço do mundo e sistema de coordenadas da câmera

O *espaço do mundo* é o sistema fixo em que a cena está descrita; as coordenadas dos pontos do arquivo VTK, depois de transformadas por $M$, estão nele. O *sistema de coordenadas da câmera* (espaço do olho) é um sistema móvel, com origem no observador, eixo $z$ apontando para trás (a câmera olha para $-z$), $x$ para a direita e $y$ para cima. Mudar de vista não move o objeto: muda a matriz que leva do mundo para a câmera. Usamos coordenadas homogêneas, vetores coluna e sistema de mão direita, como no OpenGL:

$$\mathbf{p}_{\text{clip}} = P\,V\,M\,\mathbf{p}, \qquad \mathbf{p}_{\text{NDC}} = \frac{(x_c,\, y_c,\, z_c)}{w_c}. \tag{1}$$

**Matriz de modelo.** A rotação e a escala são feitas em torno do centro $\mathbf{c}$ do modelo, para que a árvore não gire em torno da origem do mundo e saia da tela; a translação move o conjunto depois:

$$M = T(\mathbf{t})\,T(\mathbf{c})\,R_z R_y R_x\,S(s)\,T(-\mathbf{c}). \tag{2}$$

**Matriz de vista.** Equivalente ao `gluLookAt`: com o olho $\mathbf{e}$, o alvo $\mathbf{a}$ e o vetor *up* $\mathbf{u}_0$, calculam-se $\mathbf{f} = (\mathbf{a}-\mathbf{e})/\|\mathbf{a}-\mathbf{e}\|$, $\mathbf{s} = \mathbf{f}\times\mathbf{u}_0$ (normalizado) e $\mathbf{u} = \mathbf{s}\times\mathbf{f}$. As linhas da parte rotacional de $V$ são $\mathbf{s}$, $\mathbf{u}$ e $-\mathbf{f}$, e a translação é $-R\mathbf{e}$.

**Projeções.** Na ortográfica, com planos $l, r, b, t, n, f$, o volume de visão é um paralelepípedo e as distâncias não alteram o tamanho na imagem. Na perspectiva, com abertura vertical $\theta$ e razão de aspecto $a$, o volume é um tronco de pirâmide:

$$P_{\text{persp}} = \begin{pmatrix} \frac{1}{a\tan(\theta/2)} & 0 & 0 & 0 \\ 0 & \frac{1}{\tan(\theta/2)} & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}, \tag{3}$$

e a divisão por $w_c = -z_{\text{câmera}}$ faz objetos distantes parecerem menores.

**Viewport.** Para rasterizar, o NDC $[-1,1]^2$ vai para pixels de uma imagem $W \times H$, com $y$ invertido (como em imagens):

$$x_p = \tfrac{x_{\text{ndc}}+1}{2}(W-1), \qquad y_p = \tfrac{1-y_{\text{ndc}}}{2}(H-1). \tag{4}$$

### 3.4 Rasterização de linhas: Bresenham e DDA

O DDA avança um pixel no eixo de maior variação e soma um incremento fracionário no outro eixo, arredondando a cada passo. O algoritmo de Bresenham [3] chega aos pixels com aritmética inteira, mantendo um erro que mede o quanto a reta ideal se afasta do pixel escolhido. A implementação usa a forma simétrica, que funciona nos oito octantes sem trocar eixos:

$$\Delta x = |x_1 - x_0|, \quad \Delta y = -|y_1 - y_0|, \quad e_0 = \Delta x + \Delta y, \tag{5}$$

e, a cada pixel, com $e_2 = 2e$: se $e_2 \geq \Delta y$, faz-se $e \mathrel{+}= \Delta y$ e avança-se em $x$; se $e_2 \leq \Delta x$, faz-se $e \mathrel{+}= \Delta x$ e avança-se em $y$. No primeiro octante, isso é a decisão clássica com $p = 2\Delta y - \Delta x$.

### 3.5 Preenchimento de polígonos: *Scanline*

O *Scanline Fill* percorre as linhas de varredura de cima para baixo. Uma *tabela de arestas* (ET) guarda, para cada aresta não horizontal, $(y_{\min}, y_{\max}, x_{y_{\min}}, \Delta x, \Delta y)$; uma *lista de arestas ativas* (AET) mantém as arestas que cruzam a linha atual. Em cada linha, ordenam-se as interseções e preenchem-se os trechos entre pares (regra par-ímpar), o que trata polígonos côncavos. As convenções adotadas são: a varredura passa pelo centro do pixel (y inteiro); cada aresta cobre $y_{\min} \leq y < y_{\max}$ (topo exclusivo, para que um vértice não conte duas vezes); e cada trecho preenche $x_a \leq x < x_b$ (esquerda inclusiva), para que polígonos vizinhos não se sobreponham. As interseções são calculadas com frações exatas.

### 3.6 Aliasing e antialiasing

O aliasing é a "escada" que aparece quando uma reta inclinada é amostrada uma única vez por pixel. O *supersampling* (SSAA) amostra $k \times k$ vezes por pixel: a imagem é desenhada em resolução $k$ vezes maior e cada bloco $k \times k$ é reduzido à sua média (filtro de caixa). A cor final do pixel passa a ser proporcional à fração da área coberta pela linha. O MSAA, oferecido pelas GPUs, amostra a cobertura da geometria várias vezes por pixel, mas executa o sombreamento uma vez só, o que o torna mais barato que o SSAA; aqui, o SSAA é o método usado, tanto na versão própria quanto na do VTK.

### 3.7 Recorte de segmentos

**Cohen–Sutherland [8].** Cada extremo recebe um código de 4 bits (esquerda, direita, abaixo, acima). Se $c_0 \,|\, c_1 = 0$, o segmento é aceito; se $c_0 \,\&\, c_1 \neq 0$, é rejeitado. Caso contrário, um extremo exterior é levado até uma borda violada (interseção) e o código dele é recalculado, repetindo até decidir.

**Liang–Barsky [6].** Usa a forma paramétrica $P(u) = P_0 + u(P_1 - P_0)$, $0 \leq u \leq 1$. Para cada borda $k$ vale $p_k u \leq q_k$, com $p = -\Delta x,\ \Delta x,\ -\Delta y,\ \Delta y$ e $q = x_0 - x_{\min},\ x_{\max} - x_0,\ y_0 - y_{\min},\ y_{\max} - y_0$. Se $p < 0$ o segmento entra pela borda ($u_1 = \max(u_1, q/p)$); se $p > 0$ ele sai ($u_2 = \min(u_2, q/p)$); se $p = 0$, é paralelo, e é rejeitado quando $q < 0$. Rejeita-se se $u_1 > u_2$.

**Cyrus–Beck [7].** Generaliza a ideia paramétrica para qualquer janela *convexa*: para cada aresta, com normal interna $\mathbf{N}$, o ponto $P(t)$ está dentro se $\mathbf{N} \cdot (P(t) - A) \geq 0$; o sinal de $\mathbf{N} \cdot \mathbf{D}$ separa as entradas ($t_E$) das saídas ($t_L$). O Liang–Barsky é o caso particular em que as normais são os eixos.

### 3.8 Recorte de polígonos

O algoritmo de Sutherland–Hodgman [4] recorta o polígono contra *uma* aresta da janela por vez; a saída de uma etapa é a entrada da seguinte. Para cada aresta $s \to e$ do polígono, há quatro casos: ambos dentro (emite $e$); $s$ dentro e $e$ fora (emite a interseção); $s$ fora e $e$ dentro (emite a interseção e depois $e$); ambos fora (nada). Vale para janela convexa e polígono qualquer, mas, se o polígono é côncavo e o resultado correto deveria ter partes separadas, as partes saem ligadas por arestas degeneradas de largura zero, sobre a borda da janela.

O Weiler–Atherton [5] é o algoritmo adequado nesses casos. Ele percorre simultaneamente as listas de vértices do polígono e da janela, marca as interseções como de entrada ou de saída e, a partir de cada entrada, segue o polígono até a saída e então passa para a janela, fechando um contorno de cada vez. Assim, devolve vários polígonos separados quando há várias regiões, e também funciona com janelas côncavas e casos de interseção mais complexos, em que o Sutherland–Hodgman produz um único contorno com pontes degeneradas.

### 3.9 Algoritmo Desenvolvido e Arquitetura do Software

#### 3.9.1 Tecnologias

A aplicação foi escrita em Python 3.14.3. Usamos PyVista 0.49.2 e VTK 9.7.2 para a leitura dos arquivos, a janela 3D (OpenGL) e os tubos [10, 9]; NumPy 2.4.4 para as matrizes e vetores [11]; Matplotlib 3.10.8 para as figuras e o mapa de cores [12]; e PyQt5 5.15.11 com pyvistaqt 0.13.2 para a interface integrada. As versões estão fixadas no `requirements.txt`.

#### 3.9.2 Organização dos módulos

O código é dividido em módulos pequenos, cada um ligado a uma parte do enunciado (Tabela 2). Os módulos de matemática (`transformacoes.py`, `rasterizacao.py`, `recorte.py`) usam apenas NumPy e Python, sem VTK, e podem ser testados isoladamente.

#### 3.9.3 Fluxo de dados

O fluxo é: arquivo VTK → `modelo.py` (pontos, segmentos, raio) → $M$ (modelo) → $V$ (vista) → $P$ (projeção) → divisão por $w$ → viewport → pixels → *framebuffer*. Os recortes agem em coordenadas de pixel, antes de rasterizar o resultado. Para a visualização 3D interativa, a matriz $M$ é calculada por nós e entregue ao VTK; a câmera é posicionada pelas nossas vistas.

| Arquivo | Responsabilidade |
|---|---|
| `modelo.py` | leitura do VTK, extração de segmentos, detecção do atributo de raio, estatísticas |
| `transformacoes.py` | matrizes $M$, $V$, $P$, divisão perspectiva, NDC, vistas predefinidas |
| `rasterizacao.py` | Bresenham, DDA, viewport, *framebuffer*, SSAA, *Scanline Fill* |
| `recorte.py` | Cohen–Sutherland, Liang–Barsky, Cyrus–Beck, Sutherland–Hodgman |
| `visualizador.py` | janela 3D interativa, sliders, teclas, exportação de figuras |
| `didatico.py` | demonstrações e figuras dos Módulos 3 a 7 (linha de comando) |
| `main_gui.py` | interface PyQt5 que integra todos os módulos em abas |

Tabela 2: Arquitetura da aplicação.

#### 3.9.4 Observador e vistas

Foram implementadas quatro posições do observador (frontal, lateral, de topo e isométrica), definidas por uma direção a partir do alvo e um vetor *up*. A câmera fica a 2,8 vezes a diagonal do modelo do centro dele, com abertura vertical de 30°. Na ortográfica interativa, a altura visível é $h = d\tan(\theta/2)$, a mesma que a perspectiva teria no plano do alvo, o que torna as duas projeções diretamente comparáveis.

#### 3.9.5 Interface integrada

O `main_gui.py` organiza o programa em abas: *Árvore CCO* (3D interativo, sliders de translação, rotação e escala, projeção, antialiasing, cor por raio, tema, impressão de $M$, $V$ e $P$, exportação de vistas), *Bresenham* (incluindo o modo interativo por cliques), *Scanline*, *Antialiasing*, *Recorte* e *Recorte de polígonos*. Tabelas e estatísticas que sairiam no terminal aparecem no console embutido.

#### 3.9.6 Divisão entre código próprio e bibliotecas

**Implementado pelo grupo:** extração de segmentos e estatísticas; todas as matrizes e projeções; Bresenham, DDA, viewport e *framebuffer*; *supersampling* com EQM; *Scanline Fill*; Cohen–Sutherland, Liang–Barsky, Cyrus–Beck e Sutherland–Hodgman; e a orquestração, as tabelas e a interface.

**Com auxílio de bibliotecas:** PyVista/VTK leem o arquivo, desenham em OpenGL, geram os tubos, oferecem os *widgets* de slider e o antialiasing ligado por F2. Um ponto merece ser dito com precisão: a matriz $M$ é do grupo, mas é o VTK que a multiplica pelos vértices ao desenhar, e o VTK monta as suas próprias $V$ e $P$ para a janela 3D. As nossas $V$ e $P$ são usadas na rasterização, no antialiasing próprio e na figura comparativa das projeções. Matplotlib desenha as figuras didáticas, e PyQt5 fornece janela, abas e console.

## 4 Resultados e Discussão

### 4.1 Módulo 1 – Leitura, primitivas e atributos

O comando `python src/visualizador.py ARQUIVO.vtk -info` imprime as estatísticas do modelo, também exibidas no canto da janela 3D. A Tabela 3 reúne os valores dos modelos usados.

| Modelo | Pontos | Segmentos | Raio mín. | Raio máx. |
|---|---|---|---|---|
| `tree2D_Nterm008.vtk` | 16 | 15 | 0,2491 | 0,6668 |
| `tree3D_Nterm016.vtk` | 32 | 31 | 0,2261 | 0,6870 |

Tabela 3: Estatísticas dos modelos CCO carregados (saída de `-info`).

A árvore pode ser desenhada por **linhas** (espessura de tela constante) ou por **tubos** (cilindros de 16 lados cujo raio é o atributo multiplicado por uma escala visual automática, $0{,}03\,L/r_{\max}$, em que $L$ é a diagonal do modelo; a escala só serve para a visualização). Quando o raio é dado por célula, ele é interpolado para os pontos, o que o gerador de tubos exige. A cor segue o mapa *viridis* (Seção 3.2), e a tecla 7 liga ou desliga a cor por raio.

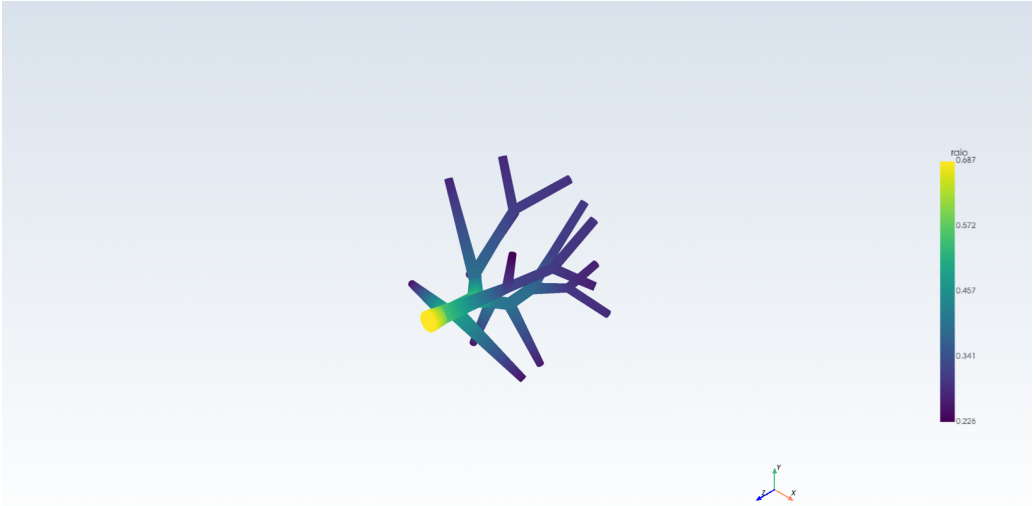



Figura 1: Árvore CCO 3D (vista isométrica, perspectiva), com tubos e cor por raio.

## 4.2 Módulo 2 – Transformações e observador

Os sliders aplicam translação (em % da diagonal), rotação em $x$, $y$ e $z$ (graus) e escala; a tecla 9 imprime $M$, $V$ e $P$. A Figura 2 mostra as quatro vistas do observador, e a Figura 3 compara as projeções ortográfica e perspectiva, calculadas apenas com as nossas matrizes, sem passar pelo VTK.

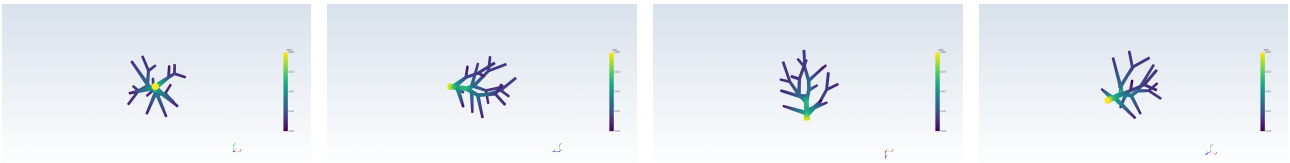


Figura 2: Quatro posições do observador (frontal, lateral, de topo e isométrica).

Na ortográfica, vasos de mesmo comprimento aparecem com o mesmo tamanho, independentemente da distância; na perspectiva, os mais distantes encolhem, o que dá noção de profundidade, mas distorce comprimentos. Para modelos 2D (planos), as duas projeções praticamente coincidem, e por isso a comparação só faz sentido nos modelos 3D.

## 4.3 Módulo 3 – Rasterização de linhas

A Figura 4 mostra o Bresenham nos oito octantes, com segmentos partindo do centro de uma grade. Para o segmento $(2,1) \to (13,6)$, com $\Delta x = 11$, $\Delta y = -5$ e $e_0 = 6$, a Tabela 4 lista os 12 pixels, número igual a $\max(|\Delta x|, |\Delta y|) + 1$, e a Figura 5 os mostra na grade.

Na validação, 1 500 segmentos aleatórios (coordenadas em $[-30, 30]$) satisfizeram: extremos corretos, exatamente $\max(|\Delta x|, |\Delta y|) + 1$ pixels, conectividade em 8 vizinhos e desvio de no máximo 0,5 pixel em relação à reta ideal. O DDA gerou um conjunto de pixels diferente do Bresenham em 273 desses segmentos; em todos eles há pelo menos um ponto em que a reta ideal passa exatamente a

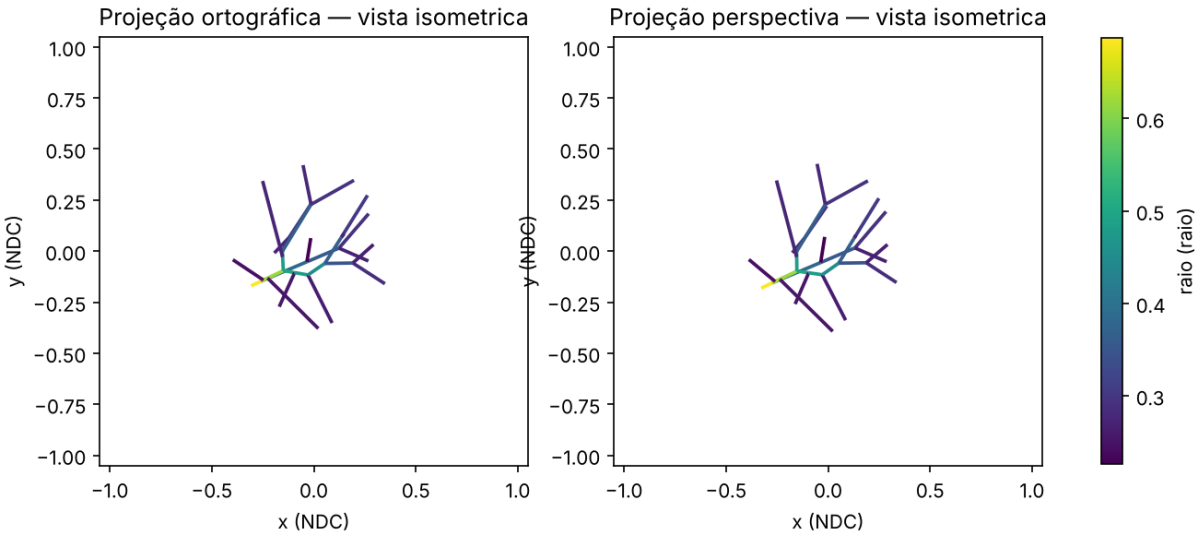



Figura 3: Projeção ortográfica (esq.) e perspectiva (dir.) da mesma árvore e da mesma vista.

| $k$ | $x$ | $y$ | $e$ | $2e$ | próximo passo |
|---|---|---|---|---|---|
| 0 | 2 | 1 | 6 | 12 | $x$ |
| 1 | 3 | 1 | 1 | 2 | $x$ e $y$ |
| 2 | 4 | 2 | 7 | 14 | $x$ |
| 3 | 5 | 2 | 2 | 4 | $x$ e $y$ |
| 4 | 6 | 3 | 8 | 16 | $x$ |
| 5 | 7 | 3 | 3 | 6 | $x$ e $y$ |
| 6 | 8 | 4 | 9 | 18 | $x$ |
| 7 | 9 | 4 | 4 | 8 | $x$ e $y$ |
| 8 | 10 | 5 | 10 | 20 | $x$ |
| 9 | 11 | 5 | 5 | 10 | $x$ e $y$ |
| 10 | 12 | 6 | 11 | 22 | $x$ |
| 11 | 13 | 6 | 6 | 12 | fim |

Tabela 4: Passo a passo do Bresenham para $(2, 1) \rightarrow (13, 6)$.




Figura 4: Bresenham nos oito octantes (linha preta: segmento ideal).

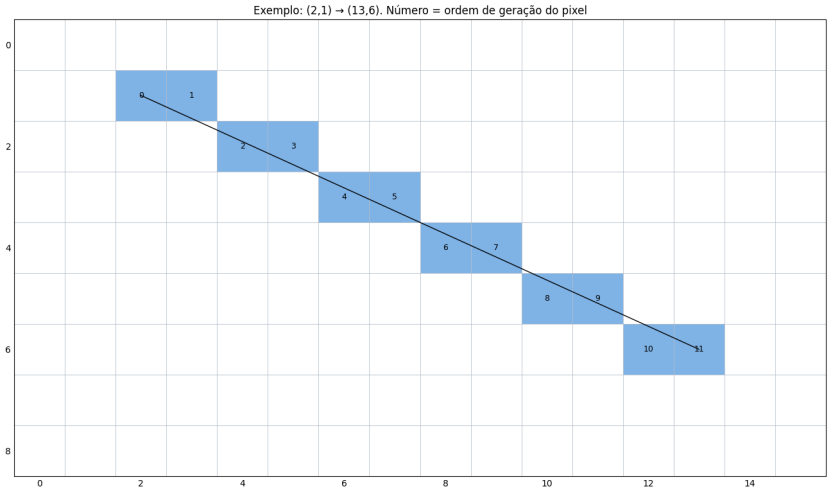



Figura 5: Pixels do exemplo numérico; o número indica a ordem de geração.

meio pixel, e os dois algoritmos desempatam de forma diferente. Sobre a árvore projetada em 2D, o comando `didatico.py bresenham -arvore ARQUIVO.vtk` rasteriza todos os segmentos (Figura 6) e informa quantos pixels diferem entre Bresenham e DDA. No modelo `tree2D_Nterm008.vtk` (imagem $600 \times 600$, vista frontal), os 15 segmentos geram 1 121 pixels em cada algoritmo, e 44 deles ficam em posições diferentes: o programa imprime 88, que é o tamanho da diferença simétrica entre os dois conjuntos (44 pixels só do Bresenham e 44 só do DDA).

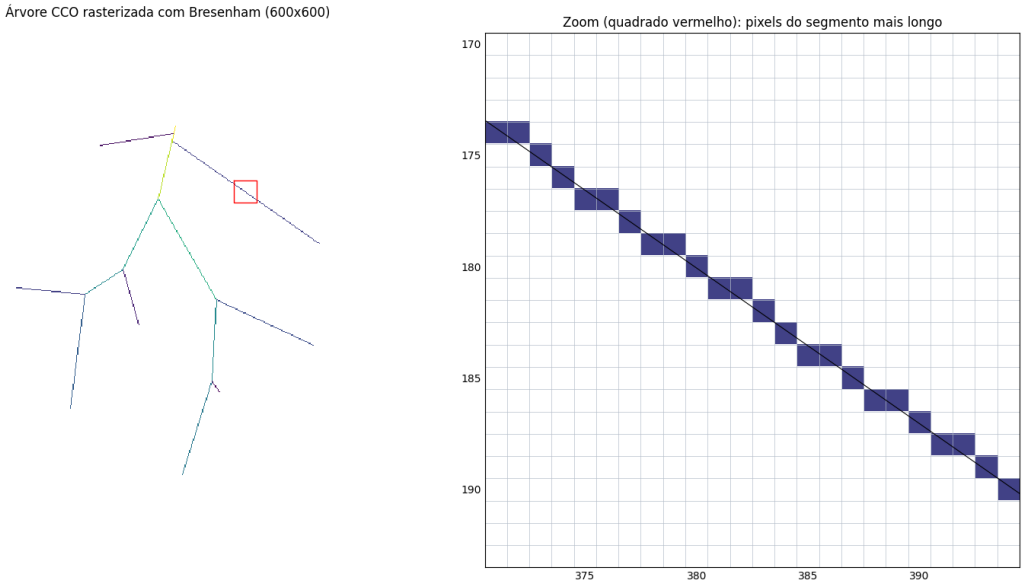



Figura 6: Árvore rasterizada com o nosso Bresenham; à direita, zoom no segmento mais longo.

### 4.4 Módulo 4 – Rasterização de polígonos

O polígono côncavo de teste, $[(2,1),(15,1),(15,11),(11,6),(7,11),(2,11)]$, tem um entalhe em V, uma aresta horizontal e um vértice em que duas arestas começam na mesma linha. A ET tem quatro arestas não horizontais (Tabela 5); a varredura (Tabela 6) mostra que, a partir de $y = 6$, a AET passa a ter quatro interseções e a linha ganha dois trechos.

O preenchimento gera 114 pixels, para uma área geométrica de 110 (fórmula do cadarço); a diferença vem da discretização: o contorno inclinado cobre pixels parcialmente. Na validação, 300

| $y_{\min}$ | $y_{\max}$ | $x$ em $y_{\min}$ | $\Delta x$ | $\Delta y$ |
|---|---|---|---|---|
| 1 | 11 | 2 | 0 | 10 |
| 1 | 11 | 15 | 0 | 10 |
| 6 | 11 | 11 | −4 | 5 |
| 6 | 11 | 11 | 4 | 5 |

Tabela 5: Tabela de arestas (ET) do polígono côncavo de teste.

| $y$ | interseções $x$ (AET ordenada) | | | | trechos preenchidos |
|---|---|---|---|---|---|
| 1–5 | 2,00 | 15,00 | | | [2, 14] |
| 6 | 2,00 | 11,00 | 11,00 | 15,00 | [2, 10] [11, 14] |
| 7 | 2,00 | 10,20 | 11,80 | 15,00 | [2, 10] [12, 14] |
| 8 | 2,00 | 9,40 | 12,60 | 15,00 | [2, 9] [13, 14] |
| 9 | 2,00 | 8,60 | 13,40 | 15,00 | [2, 8] [14, 14] |
| 10 | 2,00 | 7,80 | 14,20 | 15,00 | [2, 7] |

Tabela 6: Varredura do *Scanline Fill*: interseções e trechos por linha.

polígonos estrelados aleatórios (em geral côncavos) produziram exatamente os mesmos pixels que um teste independente de cruzamentos, que não usa ET nem AET; dois quadrados vizinhos de $10 \times 10$ geraram 100 pixels cada, sem sobreposição.

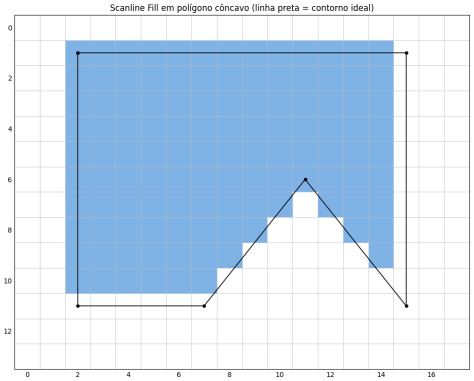



Figura 7: *Scanline Fill* no polígono côncavo (linha preta: contorno ideal).

Sobre a árvore, o polígono é uma região de interesse hexagonal preenchida com cor translúcida (Figura 8), o que liga este módulo ao anterior.

### 4.5 Módulo 5 – Aliasing e antialiasing

A comparação usa duas cenas. A primeira é um leque de retas a cada 15° mais uma reta quase horizontal, que gera degraus longos e bem visíveis, em uma imagem $64 \times 64$. A qualidade é medida pelo erro quadrático médio (EQM, escala 0–255) contra uma imagem de referência feita com supersampling de fator mais alto:

$$\mathrm{EQM}(A, B) = \frac{1}{3WH} \sum_{i,j,c} \left(A_{ijc} - B_{ijc}\right)^2. \tag{6}$$

Na cena de teste (referência $8 \times 8$), o EQM caiu de 2414,0 (uma amostra por pixel) para 54,1 (SSAA $4 \times 4$), uma redução de cerca de 98%. A imagem sem antialiasing tem só 2 cores distintas (linha e fundo); com SSAA $4 \times 4$ passam a ser 17, isto é, os 16 níveis de cobertura possíveis mais o fundo, que são os tons que suavizam a escada (Figura 9).

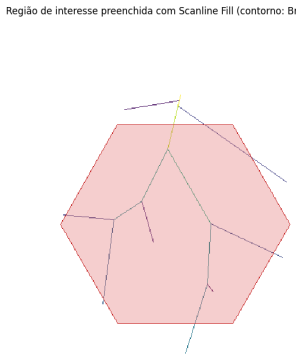



Figura 8: Região de interesse (hexágono) preenchida sobre a árvore; o contorno usa o nosso Bresenham.

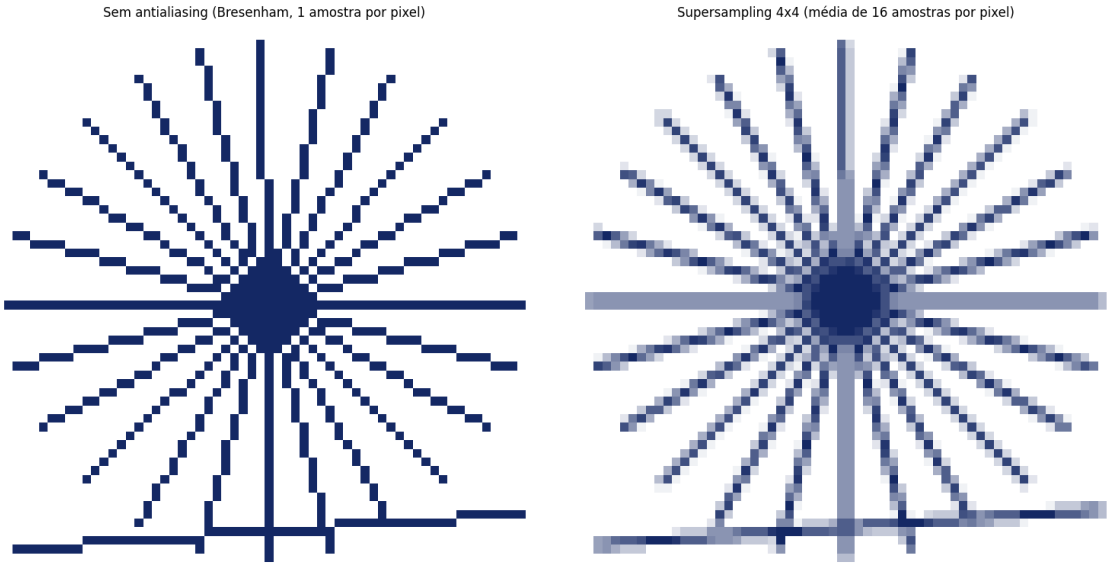



Figura 9: Cena de teste sem antialiasing (esq.) e com supersampling $4 \times 4$ (dir.).

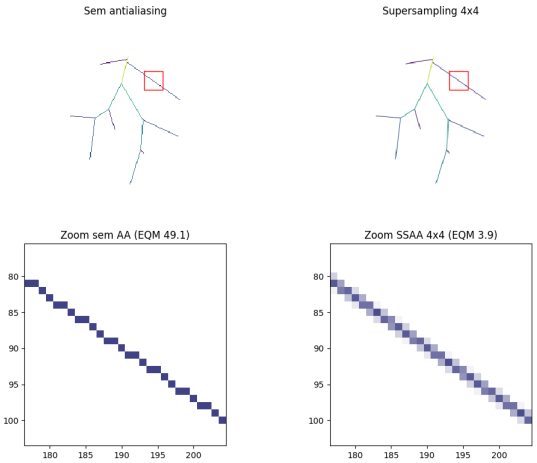



Figura 10: Árvore sem e com SSAA, com zoom no segmento mais longo e EQM de cada caso.

Sobre a árvore (Figura 10; modelo `tree2D_Nterm008.vtk`, imagem $300 \times 300$), o programa informa os EQM contra a referência (`-k 4 -ref 8`): sem AA, 49,06; SSAA $4\times4$, 3,95; redução de 92%. As cores distintas passam de 16 para 201. A comparação também pode ser vista na janela 3D do visualizador: a tecla F2 liga e desliga o SSAA do VTK, que é a técnica equivalente da biblioteca. Como a referência é, ela mesma, um supersampling, o EQM mede a convergência para esse limite e não o erro em relação à imagem ideal (ver Seção 4.9).

### 4.6 Módulo 6 – Recorte de segmentos

Os três algoritmos foram implementados e comparados; a janela é um retângulo e o resultado de cada um é o trecho visível ou nada. No exemplo numérico, $P_0 = (0, 0)$, $P_1 = (12, 10)$ e janela $x \in [2, 10]$, $y \in [2, 8]$, o Cohen–Sutherland precisa de duas interseções (Tabela 7) e o Liang–Barsky de quatro divisões, e ambos chegam ao trecho (2,40; 2,00) → (9,60; 8,00) (Tabela 8).

| it | $P_0$ | $c_0$ | $P_1$ | $c_1$ | ação |
|---|---|---|---|---|---|
| 0 | (0,00; 0,00) | 5 | (12,00; 10,00) | 10 | $P_0$ vai para a borda base → (2,40; 2,00) |
| 1 | (2,40; 2,00) | 0 | (12,00; 10,00) | 10 | $P_1$ vai para a borda topo → (9,60; 8,00) |
| 2 | (2,40; 2,00) | 0 | (9,60; 8,00) | 0 | aceita |

Tabela 7: Cohen–Sutherland no exemplo numérico (códigos: esq.=1, dir.=2, baixo=4, cima=8).

| borda | $p$ | $q$ | $r = q/p$ | $u_1$ | $u_2$ | ação |
|---|---|---|---|---|---|---|
| esquerda | −12 | −2 | 0,1667 | 0,1667 | 1,0000 | entrada: $u_1 = \max(u_1, r)$ |
| direita | 12 | 10 | 0,8333 | 0,1667 | 0,8333 | saída: $u_2 = \min(u_2, r)$ |
| base | −10 | −2 | 0,2000 | 0,2000 | 0,8333 | entrada |
| topo | 10 | 8 | 0,8000 | 0,2000 | 0,8000 | saída |

Tabela 8: Liang–Barsky no mesmo exemplo: $u_1 = 0{,}2$ e $u_2 = 0{,}8$.

A cena de teste tem dez segmentos e a janela $x \in [4, 14]$, $y \in [3, 9]$ (Figura 11). Pelo Cohen–Sutherland, 1 foi aceito direto, 3 foram rejeitados direto, 5 foram recortados e 1 foi rejeitado só depois de recortado: o segmento $(2, 8) \to (5, 10{,}5)$ passa fora do canto superior esquerdo, e isso só se descobre depois da primeira interseção. Os três algoritmos deram o mesmo resultado nos dez.

Sobre a árvore (Figura 12; modelo `tree2D_Nterm008.vtk`, imagem $300 \times 300$, janela 0,25–0,75 da imagem, isto é, $x, y \in [75, 225]$), o programa conta os casos e as operações: 7 segmentos aceitos direto, 3 rejeitados direto, 5 recortados e nenhum rejeitado após recortar; Cohen–Sutherland calculou 6 interseções e Liang–Barsky fez 57 divisões; os resultados são idênticos nos três algoritmos em 15 de 15 segmentos. Como 10 dos 15 segmentos se decidem só pelos códigos de região, o Cohen–Sutherland faz bem menos contas aqui; o Liang–Barsky divide em quase todas as bordas de todos os segmentos (57 de no máximo $15 \times 4 = 60$).

**Os três algoritmos.** O Cohen–Sutherland é barato quando a maioria dos segmentos se decide pelos códigos (aceitação ou rejeição trivial), mas pode calcular interseções que depois se revelam inúteis, como no segmento 9. O Liang–Barsky usa apenas comparações de $p$ e $q$ e faz no máximo uma divisão por borda, calculando as interseções uma vez só, no final. O Cyrus–Beck faz o mesmo raciocínio paramétrico, mas com normais e produtos escalares, o que o torna o único dos três que serve para janelas convexas arbitrárias, ao custo de mais operações por borda.

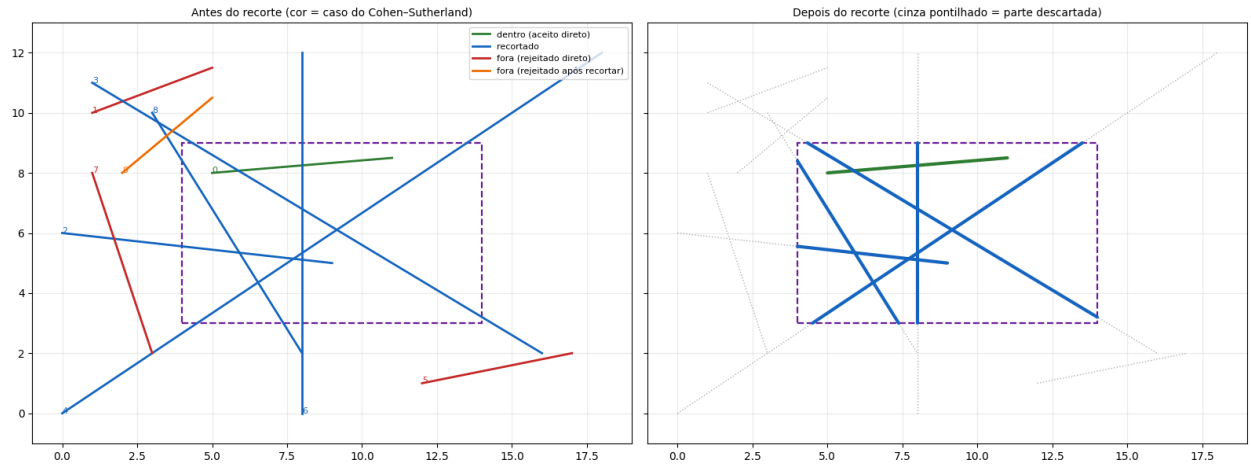



Figura 11: Cena de teste antes (esq.) e depois (dir.) do recorte; a cor indica o caso do Cohen–Sutherland.

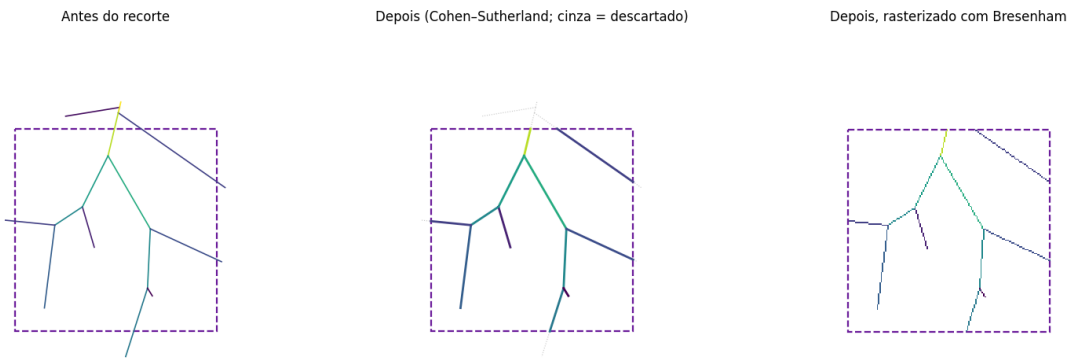



Figura 12: Recorte da árvore: antes, depois (cinza: descartado) e rasterizado com Bresenham.

### 4.7 Módulo 7 – Recorte de polígonos

O Sutherland–Hodgman foi implementado para janela convexa qualquer (e não só retangular) e testado em dois polígonos. O primeiro, convexo, tem vértices $[(1,2),(9,0),(13,6),(8,11),(2,9)]$ e a janela é $x \in [3,11]$, $y \in [3,8]$. A Tabela 9 mostra o resultado após cada borda; a área vai de 88 para 40.

| Borda | Vértices | Polígono resultante |
|---|---|---|
| base | 5 | (1,14; 3) (11; 3) (13; 6) (8; 11) (2; 9) |
| direita | 5 | (1,14; 3) (11; 3) (11; 8) (8; 11) (2; 9) |
| topo | 4 | (1,86; 8) (1,14; 3) (11; 3) (11; 8) |
| esquerda | 4 | (3; 8) (3; 3) (11; 3) (11; 8) |

Tabela 9: Sutherland–Hodgman passo a passo no polígono convexo (uma borda por vez).

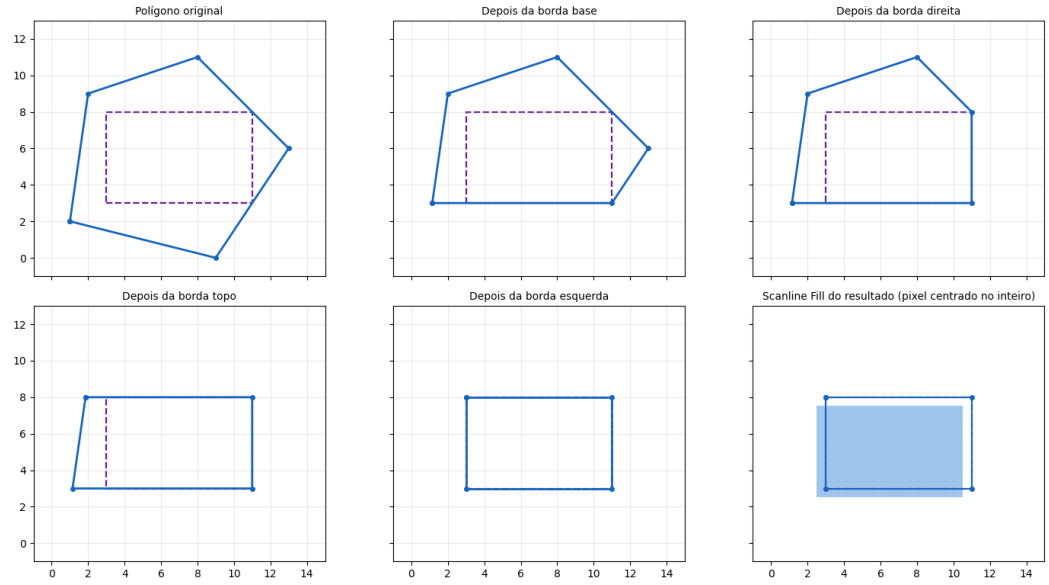



Figura 13: Etapas do Sutherland–Hodgman e preenchimento do resultado com o nosso *Scanline*.

O segundo polígono é côncavo, em forma de U, com vértices

$$(1,1),\ (17,1),\ (17,12),\ (12,12),\ (12,5),\ (6,5),\ (6,12),\ (1,12),$$

recortado pela janela $x \in [2,16]$, $y \in [6,10]$. O resultado correto são *dois* retângulos separados, os dois braços do U, de área $16+16=32$. O algoritmo devolve, de fato, a área 32, mas como um único polígono de 8 vértices, em que os dois retângulos são ligados por um trecho degenerado de $(6,6)$ a $(12,6)$, percorrido nos dois sentidos (em vermelho na Figura 14). Quando o resultado só é desenhado com preenchimento, o defeito passa despercebido, mas, ao desenhar o contorno ou ao usar o polígono em outras operações, a aresta de largura zero aparece. Esse é o caso em que o Weiler–Atherton é o algoritmo adequado, pois entregaria dois polígonos distintos.

Sobre a árvore (Figura 15), uma estrela de cinco pontas é recortada e o resultado é preenchido com o *Scanline* e contornado com o Bresenham do grupo, integrando os módulos 3, 4 e 7.

### 4.8 Autotestes

Os módulos de rasterização e de recorte têm autotestes executáveis (`python src/rasterizacao.py` e `python src/recorte.py`). Os resultados estão na Tabela 10.

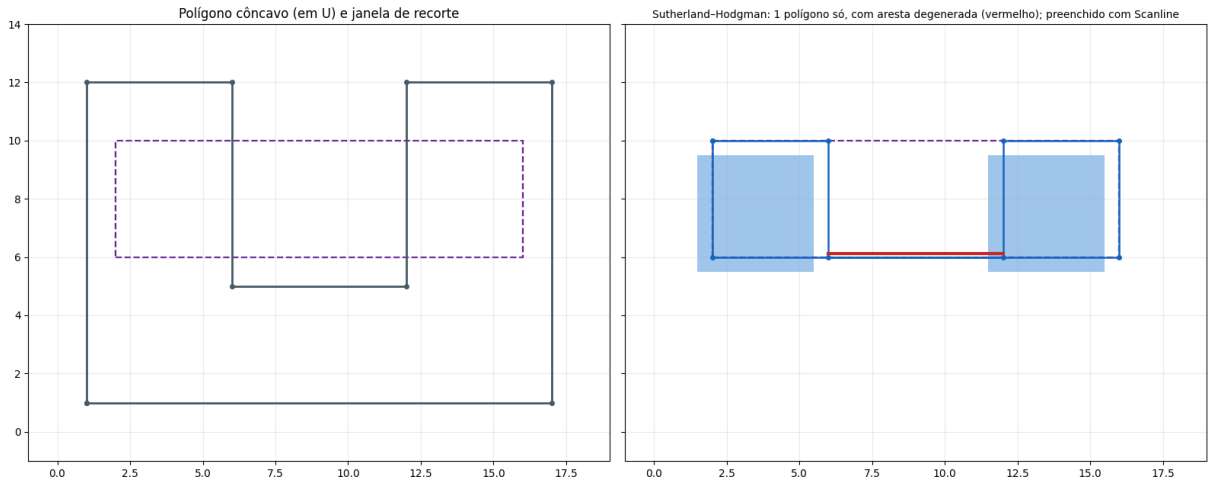



Figura 14: Polígono em U e janela (esq.); resultado do Sutherland–Hodgman com a aresta degenerada em vermelho (dir.).

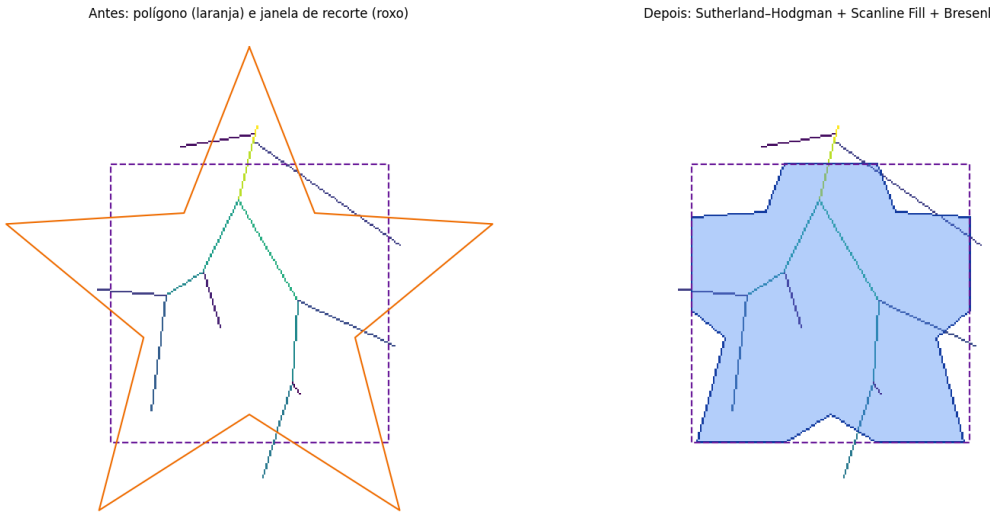



Figura 15: Estrela côncava recortada sobre a árvore: antes (esq.) e depois (dir.), com Sutherland–Hodgman, *Scanline* e Bresenham.

| Teste | Resultado |
|---|---|
| Bresenham | 1 500 segmentos aleatórios: extremos, número de pixels, conectividade e desvio $\leq 0{,}5$ |
| *Scanline* | 300 polígonos aleatórios iguais ao teste independente de cruzamentos; vizinhos sem sobreposição |
| Recorte de segmentos | 20 000 pares segmento/janela: Cohen–Sutherland, Liang–Barsky e Cyrus–Beck coincidem; trechos dentro da janela e sobre a reta original |
| Sutherland–Hodgman | 600 polígonos (côncavos incluídos) $\times$ janelas aleatórias, 60 pontos cada: região resultante correta; quadrado $4 \times 4$ contra janela deslocada: área 4; janela disjunta: resultado vazio |

Tabela 10: Resumo dos autotestes.

### 4.9 Discussão dos Achados

**Decisões de implementação.** Os módulos de matemática foram separados do VTK, para que cada algoritmo possa ser lido, testado e demonstrado sem caixa-preta. O custo é que a janela 3D interativa e a rasterização própria usam câmeras construídas por caminhos diferentes: a primeira pelo VTK, a segunda pelas nossas matrizes. Para reduzir essa diferença, a posição da câmera do VTK vem das nossas vistas e a ortográfica interativa usa a mesma altura visível da perspectiva no plano do alvo.

**Bresenham e DDA.** O Bresenham evita ponto flutuante e as divisões, e produz os mesmos pixels em qualquer octante graças à forma simétrica. O DDA, mais simples, difere em uma parte dos segmentos (273 de 1 500 no teste), e em todos eles há pontos de empate (meio pixel). Nenhum dos dois gera resultado visualmente errado; a preferência pelo Bresenham é por precisão numérica e custo.

**Antialiasing.** O SSAA suaviza a escada à custa de $k^2$ vezes mais amostras (o tempo e a memória crescem com $k^2$). Como a referência do EQM é um supersampling de fator 8, a métrica mede a convergência para esse limite, não o erro em relação à reta ideal; ainda assim, ela ordena corretamente as imagens. O filtro de caixa é o mais simples e pode deixar bordas um pouco menos suaves que filtros maiores.

**Recorte de segmentos.** Os três algoritmos concordaram em todos os casos testados, exceto nos degenerados em que o segmento apenas toca um canto da janela, que o autoteste ignora de propósito. Na prática, o Cohen–Sutherland é vantajoso quando a maior parte dos segmentos está longe da janela, como no recorte de uma pequena região da árvore; o Liang–Barsky é mais regular; e o Cyrus–Beck é a escolha quando a janela não é um retângulo alinhado aos eixos.

**Recorte de polígonos.** O Sutherland–Hodgman é simples e eficiente, mas tem a limitação demonstrada na Figura 14. O Weiler–Atherton resolve isso e também aceita janelas côncavas; sua implementação completa não foi feita e fica como possível extensão.

**Limitações.** (i) A rasterização própria usa projeção ortográfica e não resolve visibilidade (não há *z-buffer*), assunto do TP2; (ii) os vértices do *Scanline* são inteiros, e por isso os polígonos vindos do recorte (em ponto flutuante) são arredondados antes de preencher; (iii) o recorte de segmentos e polígonos é 2D, em coordenadas de pixel; (iv) o antialiasing do VTK (F2) é usado como está, sem controle do fator.

## 5 Conclusão

O trabalho integrou, em uma aplicação única, os módulos pedidos: leitura e visualização dos modelos CCO com cor por raio; transformações, vistas e as projeções ortográfica e perspectiva; Bresenham, *Scanline* e *supersampling*; Cohen–Sutherland, Liang–Barsky e Cyrus–Beck; e Sutherland–Hodgman. Os algoritmos foram escritos a partir da matemática e conferidos por testes aleatórios, e o VTK foi usado para a leitura do arquivo, o desenho OpenGL, os tubos e, na janela 3D interativa, a montagem das matrizes de vista e projeção a partir das nossas câmeras; os algoritmos pedidos pelo enunciado são do grupo. O caso do polígono em U mostrou na prática por que o Weiler–Atherton existe, e o código está pronto para ser estendido com ele. Como próximos passos, o TP2 poderá usar a infraestrutura de matrizes e rasterização para incorporar visibilidade, iluminação e técnicas baseadas em raios.

## Agradecimentos

Os autores agradecem ao Departamento de Computação (DECOM) e à Universidade Federal de Ouro Preto (UFOP) pela infraestrutura e pelo suporte institucional, e ao Prof. Dr. Rafael Alves Bonfim de Queiroz pelos modelos CCO fornecidos e pelo enunciado do trabalho.

## Referências

[1] SCHREINER, W.; BUXBAUM, P. Computer-optimization of vascular trees. **IEEE Transactions on Biomedical Engineering**, v. 40, n. 5, p. 482–491, 1993.

[2] KARCH, R.; NEUMANN, F.; NEUMANN, M.; SCHREINER, W. A three-dimensional model for arterial tree representation, generated by constrained constructive optimization. **Computers in Biology and Medicine**, v. 29, n. 1, p. 19–37, 1999.

[3] BRESENHAM, J. E. Algorithm for computer control of a digital plotter. **IBM Systems Journal**, v. 4, n. 1, p. 25–30, 1965.

[4] SUTHERLAND, I. E.; HODGMAN, G. W. Reentrant polygon clipping. **Communications of the ACM**, v. 17, n. 1, p. 32–42, 1974.

[5] WEILER, K.; ATHERTON, P. Hidden surface removal using polygon area sorting. **ACM SIGGRAPH Computer Graphics**, v. 11, n. 2, p. 214–222, 1977.

[6] LIANG, Y.-D.; BARSKY, B. A. A new concept and method for line clipping. **ACM Transactions on Graphics**, v. 3, n. 1, p. 1–22, 1984.

[7] CYRUS, M.; BECK, J. Generalized two- and three-dimensional clipping. **Computers & Graphics**, v. 3, n. 1, p. 23–28, 1978.

[8] FOLEY, J. D.; VAN DAM, A.; FEINER, S. K.; HUGHES, J. F. **Computer Graphics: Principles and Practice**. 2. ed. Reading: Addison-Wesley, 1990.

[9] SCHROEDER, W.; MARTIN, K.; LORENSEN, B. **The Visualization Toolkit**. 4. ed. Kitware, 2006.

[10] SULLIVAN, C. B.; KASZYNSKI, A. A. PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK). **Journal of Open Source Software**, v. 4, n. 37, p. 1450, 2019.

[11] HARRIS, C. R. et al. Array programming with NumPy. **Nature**, v. 585, p. 357–362, 2020.

[12] HUNTER, J. D. Matplotlib: a 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.